

ADA Compliance Scanner (an ASP.NET Core 10 app created by Roger Grajeda).

Summary of the main features focused on what the application currently does and what has been tested.

The overall goal of the City Website Audit Dashboard is to move accessibility testing beyond simply scanning individual public URLs. It can audit public pages as well as authenticated, multi-step application workflows, save the results, organize the findings, and track issues through remediation and retesting.

Main Features

- **Public website accessibility scanning:** Individual public URLs can be scanned and stored on the dashboard. There is also a Batch Analysis feature for scanning multiple URLs at once, with optional WAVE accessibility testing.
- **Authenticated application auditing:** The application can audit pages that only become available after login or after navigating through a workflow. The user signs in normally, and the same authenticated browser session is maintained while testing different application states.
- **Playwright browser automation:** Playwright controls a real Microsoft Edge browser. This allows the dashboard to interact with the actual rendered application rather than only downloading the initial HTML of a page. It can follow multi-step workflows, work with JavaScript-driven pages, preserve authentication, detect new browser states, and interact with real page controls.
- **axe-core accessibility testing:** Each authenticated workflow state can be tested with axe-core. The dashboard stores violations, items needing manual review, severity, WCAG tags/levels, affected element counts, and the exact affected elements.
- **Element-level repair guidance:** For affected elements, the application saves the selector/HTML, the reason the element failed, and practical “How to fix this exact element” guidance to make the findings more useful to developers.
- **Manual and automatic workflow modes:** Users can scan a page manually, analyze the current state, fill supported fields, preview the next automatic action, advance one step, or run a larger automatic workflow.
- **Automatic form navigation:** The workflow automation can fill ordinary test fields, select BOE address autocomplete results, handle standard Yes/No controls,

navigate workflow hubs, visit required and optional sections, and handle supported file-upload pages using harmless generated test files.

- **Safety controls:** The automation deliberately avoids irreversible or sensitive actions. Final submission, payment, certification, signatures, consent, CAPTCHA, banking/SSN fields, and similar actions are left for manual review. The user remains in control and can stop or resume an automated workflow.
- **Dynamic workflow handling:** The scanner detects changes in the rendered page even when the URL does not change. It also detects repeated states, avoids workflow loops, retries certain dynamic pages when necessary, and prevents the same workflow state from being repeatedly recorded.
- **Audit history:** Every authenticated audit can be saved in SQL Server, including the workflow states that were scanned and the accessibility findings discovered on each state.
- **Detailed audit results:** Audit Details shows the scanned states, violations, manual-review findings, WCAG information, severity, affected elements, and prioritized “Fix First” findings.
- **PDF reporting:** Authenticated audit results can be exported into a PDF report using the same saved audit data shown in the dashboard.
- **Accessibility Overview:** A management-level dashboard summarizes authenticated accessibility results across applications. It includes filters, total applications audited, states scanned, findings, affected elements, high-priority findings, automated test-pass information, trends, recurring findings, and comparisons between audits.
- **Remediation tracking:** A finding can be turned into a remediation work item, assigned to someone, given a status, and accompanied by notes and history. The lifecycle is essentially **Scan → Detect → Assign → Remediate → Retest → Verify → Report Progress**.
- **Retesting and regression detection:** A tracked issue can be retested against the current application state or compared against a later full workflow audit. If an issue that was previously considered fixed appears again, the system can reopen it for further work.
- **Verification safeguards:** “Fixed” and “Verified” are intentionally separate. A user cannot simply mark an issue as Verified without supporting retest evidence. This

preserves an audit trail and avoids treating an automated result as an automatic compliance decision.

- **Reusable workflow approach:** The scanner was built to recognize common application patterns rather than being hard-coded to only one page. We have successfully tested representative workflows in A-Permits and CSR, and the most extensive completed workflow test so far has been R-Permit, which included required and optional sections, address selection, Yes/No decisions, required and optional attachments, returning to the workflow hub, loop prevention, and safely stopping before final submission. Additional permit types and workflow branches still need broader testing before we can say every possible permit path is supported.

Role of Playwright

The easiest way to explain Playwright is that it allows the accessibility application to behave like a real user operating a real browser. Instead of only scanning the first URL of an application, it can keep the login session alive, move through application screens, interact with JavaScript controls, and test pages that would otherwise be inaccessible to a traditional URL scanner.

axe-core then performs the accessibility analysis on each rendered browser state. Playwright handles the browser/workflow, axe-core handles the accessibility rules.

Main Advantage of the Application

I think the biggest advantage is that it brings several normally separate activities into one system.

A traditional accessibility scanner can tell us that a public page has issues. This application can go further by auditing actual authenticated application workflows, preserving the findings for every state, identifying the exact affected elements, helping developers understand how to correct them, tracking those issues through remediation, retesting them later, and showing management the progress

A short way to describe it would be:

“The application goes beyond scanning individual URLs. It can audit the actual authenticated workflow of an application, preserve the findings from each state, and track accessibility issues from detection through remediation and retesting.”

Current Status / Important Context

The application is still being prepared for broader staging testing. We should not say that every possible permit workflow has already been tested. The automation is intentionally

designed so that unusual controls or new workflow patterns can stop for manual intervention rather than making unsafe assumptions.

The dashboard/reporting/database portion has also been successfully tested running under IIS with SQL Server.

Notes from Roger Grajeda: *“The one deployment item I am currently finishing is the interactive Playwright browser portion. A visible Edge browser works correctly when the application runs in a logged-in desktop session, but IIS itself runs in a non-interactive Windows session. I am separating that browser automation into a small workstation-side process so the IIS-hosted dashboard can communicate with an interactive Playwright/Edge session. This should preserve the same experience we currently have when running the application locally.*

The current authenticated audit supports one active browser audit session at a time. Multi-user simultaneous authenticated auditing is still something to be added.”